

UNIVERSIDAD DEL VALLE DE GUATEMALA



Lab1 - Introducción Wireshark

Brandon Werner Rivera Cabrera 23088

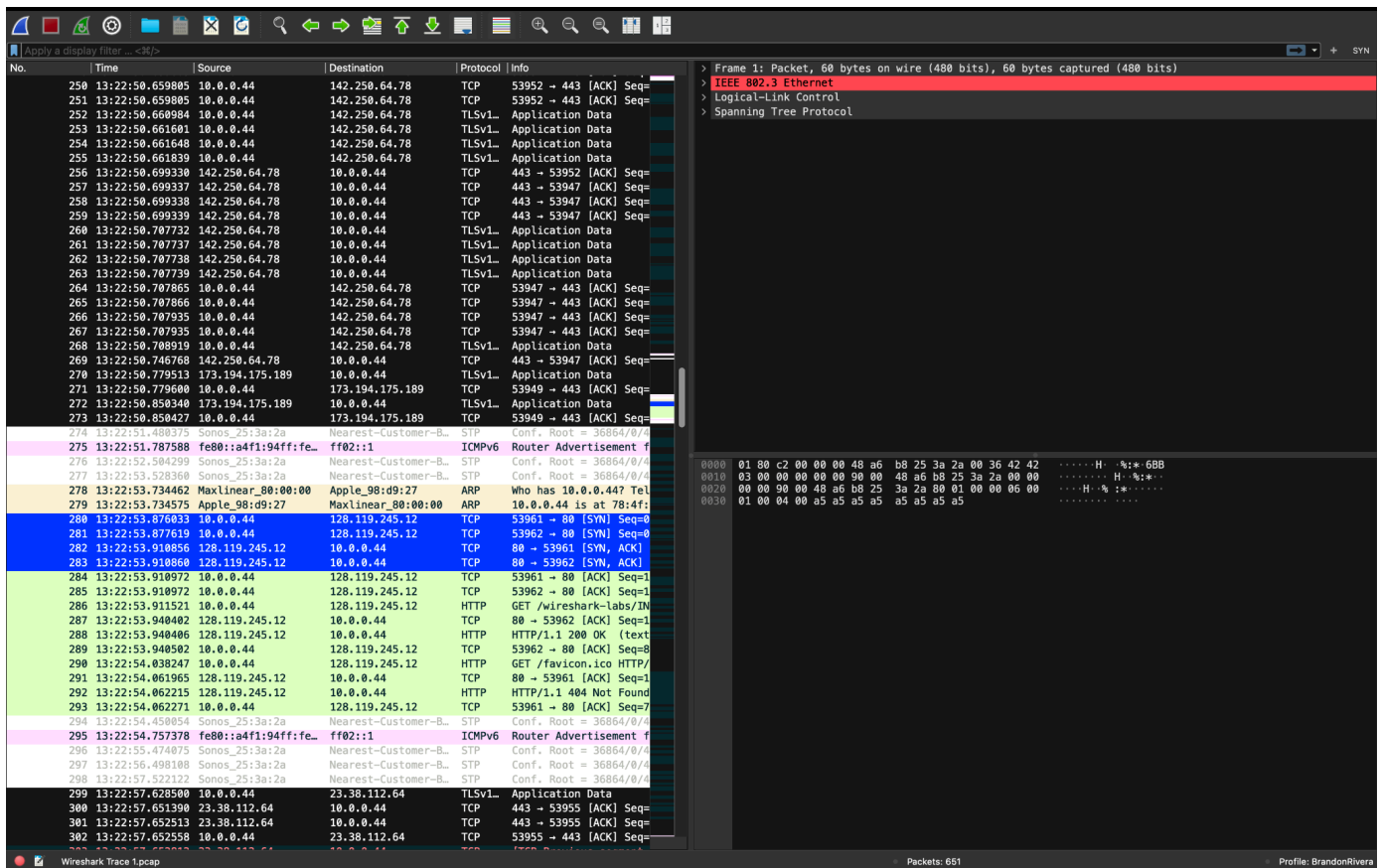
GUATEMALA
13 de julio de 2026

Se debe descargar e instalar el software de Wireshark. Es probable que para ejecutarlo pida permisos de administrador (sudo, click + run as admin, etc.).

3.4 Personalización del entorno

- Inicie Wireshark
- Cree un perfil con su primer nombre y primer apellido (edit -> configuration profile)
- Descargue el archivo .pcap proporcionado en Canvas: intro-wireshark-trace1.pcap
- Abra el archivo descargado, el archivo contiene transmisiones capturadas, y existen diversas columnas que representan la data.
- Aplique el formato de tiempo Time of Day (view -> Time Display Format)
- Agregue una columna con la longitud del protocolo (preferences -> column -> +)
- Elimine u oculte la columna Longitud (click derecho -> desmarcar columna)
- Aplique un esquema de paneles que sea de su preferencia (que no sea el esquema por defecto) (preferences -> Layout)
- Aplique una regla de color para el protocolo TCP cuyas banderas SYN sean iguales a 1, y coloque el color de su preferencia. (View -> coloring rules -> +)
- Cree un botón que aplique un filtro para paquetes TCP con la bandera SYN igual a 1. (esquina superior derecha -> +)
- Oculte las interfaces virtuales (en caso aplique: capture -> options)

Se debe realizar tomas de pantalla que muestren el entorno final personalizado, el nombre del perfil y el uso de las reglas de color y botón del filtro, así como la lista simplificada de las interfaces de captura.



3.5 Configuración de la captura de paquetes

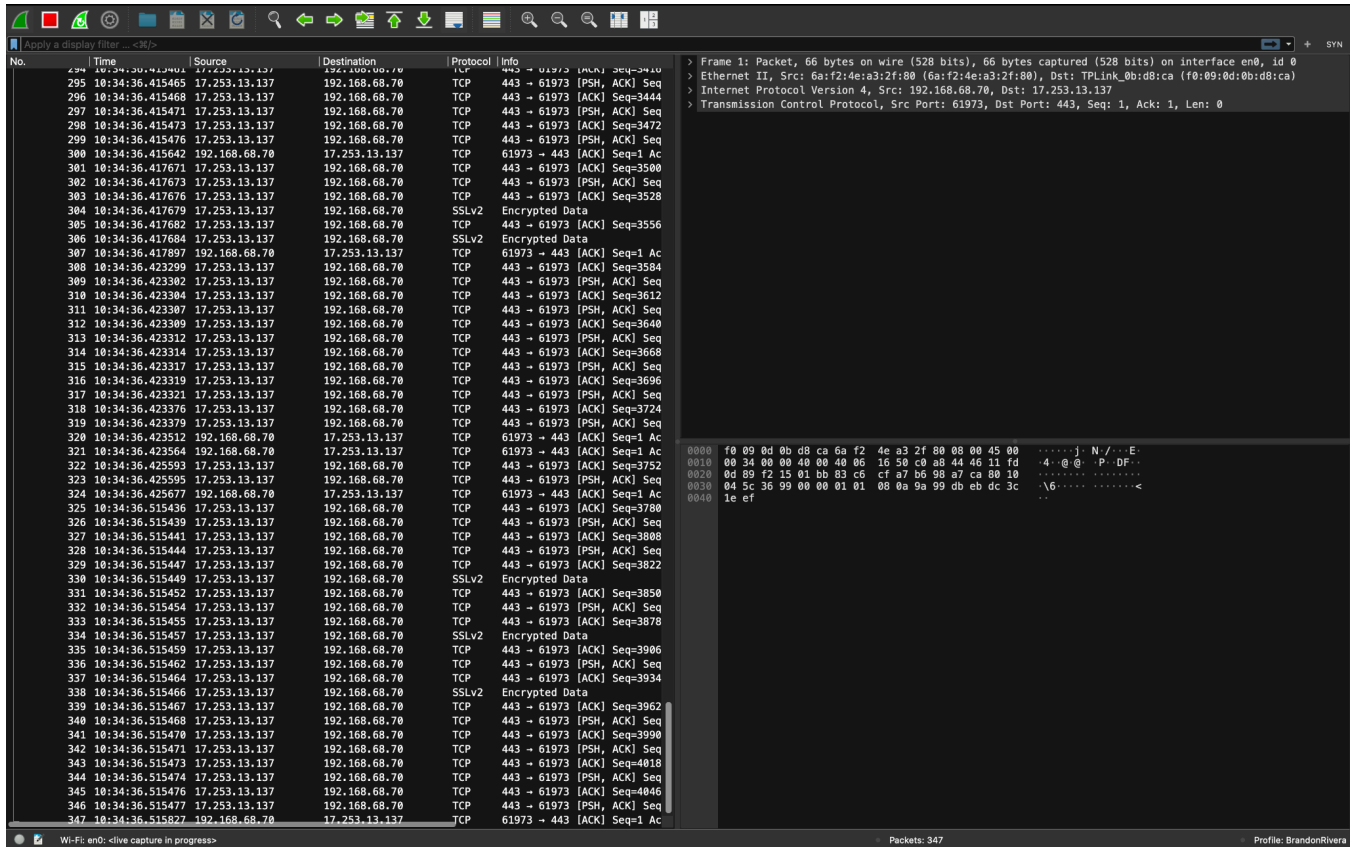
En la segunda parte, se realizará una captura de paquetes con un ring buffer.

- Abra una terminal y ejecute el comando `ifconfig/ipconfig` (dependiendo de su OS). Detalle y explique lo observado, investigue (i.e.: 'man ifconfig', documentación) de ser necesario.

```
> ifconfig
lo0: flags=8049<UP,LOOPBACK,RUNNING,MULTICAST> mtu 16384
    options=1203<RXCSUM,TXCSUM,TXSTATUS,SW_TIMESTAMP>
    inet 127.0.0.1 netmask 0xff000000
    inet6 ::1 prefixlen 128
    inet6 fe80::1%lo0 prefixlen 64 scopeid 0x1
    nd6 options=201<PERFORMNUD,DAD>
gif0: flags=8010<POINTOPOINT,MULTICAST> mtu 1280
stf0: flags=0<> mtu 1280
anpi0: flags=8863<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST> mtu 1500
    options=400<CHANNEL_IO>
    ether b2:97:6b:f3:02:e1
    media: none
    status: inactive
anpi1: flags=8863<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST> mtu 1500
    options=400<CHANNEL_IO>
    ether b2:97:6b:f3:02:e2
    media: none
    status: inactive
en3: flags=8863<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST> mtu 1500
    options=400<CHANNEL_IO>
    ether b2:97:6b:f3:02:c1
    nd6 options=201<PERFORMNUD,DAD>
    media: none
    status: inactive
en4: flags=8863<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST> mtu 1500
    options=400<CHANNEL_IO>
    ether b2:97:6b:f3:02:c2
    nd6 options=201<PERFORMNUD,DAD>
    media: none
    status: inactive
en1: flags=8963<UP,BROADCAST,SMART,RUNNING,PROMISC,SIMPLEX,MULTICAST> mtu 1500
    options=460<TS04,TS06,CHANNEL_IO>
    ether 36:b5:48:e0:81:80
    media: autoselect <full-duplex>
    status: inactive
en2: flags=8963<UP,BROADCAST,SMART,RUNNING,PROMISC,SIMPLEX,MULTICAST> mtu 1500
    options=460<TS04,TS06,CHANNEL_IO>
    ether 36:b5:48:e0:81:84
    media: autoselect <full-duplex>
    status: inactive
bridge0: flags=8863<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST> mtu 1500
    options=63<RXCSUM,TXCSUM,TS04,TS06>
```

- lo0 (loopback):
 - la interfaz de loopback estándar, usada para tráfico interno de la máquina hacia sí misma. Siempre está UP y RUNNING, y su MTU es alta (16384) porque no pasa por hardware físico, solo por memoria/kernel.
- en0:
 - esta es la interfaz de Wi-Fi, y es la única con status, active y con una IP real asignada por DHCP: 192.168.68.70. Esta es la interfaz que realmente es la utilizada para navegar.
- Luego, retornando a Wireshark, desactive las interfaces virtuales o que no aplique.
- Realice una captura de paquetes con la interfaz de Ethernet o WiFi con una configuración de ring buffer, con un tamaño de 5 MB por archivo y un número máximo de 10 archivos (puede hacerlo por medio de la interfaz de usuario o por medio de comandos) Genere tráfico para que los archivos se creen. Defina el nombre de los archivos de la siguiente forma: lab1_carnet.pgcap (options -> capture -> output)

Se debe realizar tomas de pantalla de la configuración o comandos para la creación del ring buffer, así como los archivos generados.



3.6 Análisis de paquetes

En la tercera parte se analizará el protocolo HTTP. Debe realizar tomas de pantalla que validen sus respuestas.

- Abra su navegador, inicie una captura de paquetes en Wireshark (sin filtro) en la interfaz y acceda a la siguiente dirección: <http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html>
- Detenga la captura de paquetes (si desea realizar una nueva captura de la página deberá borrar el caché de su navegador, de lo contrario no se realizará la captura del protocolo HTTP).
- Responda las siguientes preguntas:
 - ¿Qué versión de HTTP está ejecutando su navegador?

Destination	Protocol	Info	Protocol Length
128.119.245.12	HTTP	GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1	678

i. HTTP 1.1

- ¿Qué versión de HTTP está ejecutando el servidor?

HTTP/1.1 200 OK (text/html)

i. HTTP 1.1

- ¿Qué lenguajes (si aplica) indica el navegador que acepta a el servidor?

i. en-US,en;q=0.9

- ¿Cuántos bytes de contenido fueron devueltos por el servidor?

Info	Protocol Length
GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1	567
HTTP/1.1 200 OK (text/html)	507

- e. En el caso de que haya un problema de rendimiento mientras se descarga la página, ¿en que elementos de la red convendría “escuchar” los paquetes? ¿Es conveniente instalar Wireshark en el servidor? Justifique.
 - i. Si hay un problema de rendimiento al descargar una página, conviene capturar tráfico en varios puntos de la red para aislar dónde está el cuello de botella: en el cliente, en el gateway/router de una red local, y si es posible, en algún punto intermedio de la red del proveedor de internet (ISP) o en el borde de la red del servidor, para comparar tiempos de tránsito.
 - ii. Wireshark no es siempre conveniente porque muchas veces no tiene acceso a toda la información del servidor, también cuando captura el tráfico en un servidor de producción anade carga de procesamiento lo que empeora el rendimiento.

Discussion

La primera parte de la práctica permitió familiarizarnos con el entorno de Wireshark más allá de su uso básico de captura. Un hallazgo particular fue que, al agregar una columna para representar la longitud del protocolo, no existe un tipo de columna llamado explícitamente “Protocol Length”; la opción correcta es “Packet length (bytes)”, ya construida en Wireshark.

Otro hallazgo relevante surgió al ejecutar `ifconfig` antes de iniciar la captura. En una computadora macOS moderna aparecen más de una decena de interfaces de red, pero la gran mayoría no transportan tráfico de usuario: `lo0` es la interfaz de loopback, `gif0` y `stf0` son interfaces de túnel para IPv6, `awdl0` y `llw0` corresponden a Apple Wireless Direct Link (usadas por AirDrop, Handoff y funciones similares), los `utun0-utun7` son túneles virtuales asociados a servicios como iCloud Private Relay o VPNs, y `en1/en2/en3/en4` junto con `bridge0` son interfaces relacionadas con Thunderbolt Bridge. De todas ellas, únicamente `en0` (WiFi) mostró una dirección IP real asignada por DHCP y estado “active”, confirmando que es la única interfaz relevante para la captura de tráfico.

Durante la configuración del ring buffer (archivos de 5 MB, máximo 10 archivos) surgió un obstáculo práctico: en macOS, Wireshark no tenía permisos para abrir el dispositivo BPF (`/dev/bpf0`), lo cual impidió iniciar la captura en la interfaz `en0`. La solución fue instalar la utilidad `ChmodBPF` incluida con Wireshark y reiniciar la sesión del sistema para que el usuario quedara agregado al grupo `access_bpf`.

La segunda parte, centrada en capturar una petición HTTP real hacia `gaia.cs.umass.edu`, resultó más compleja de lo esperado debido al comportamiento del navegador. En los primeros intentos, la página cargó visualmente, pero al filtrar por `http.request` no aparecía ninguna petición dirigida al dominio objetivo, lo que indicaba que el navegador había servido el contenido directamente desde caché sin comunicarse con el servidor. En un segundo intento, sí se generó una petición, pero el servidor respondió con HTTP/1.1 304 Not Modified en lugar de 200 OK; esto ocurre cuando el navegador envía una petición condicional y el servidor confirma que el recurso no ha cambiado, por lo que no reenvía el cuerpo del contenido. Aunque un 304 sigue siendo válido para identificar la versión de HTTP utilizada, no permite responder con precisión cuántos bytes de contenido fueron devueltos, ya que ese campo no viene presente o viene en cero. Para forzar una respuesta 200 OK completa se intentó usar el modo incógnito de Chrome, lo cual introdujo un segundo obstáculo: Chrome activa por defecto el modo “Always use secure connections” (HTTPS-First) dentro de ventanas de incógnito, redirigiendo automáticamente la conexión a HTTPS aunque la URL solicitada fuera explícitamente `http://`. La solución más confiable fue borrar por completo el caché del navegador y repetir la captura en una ventana normal, evitando así tanto el caché persistente como la redirección forzada.

Una vez obtenida una respuesta 200 OK genuina, fue posible confirmar que tanto el navegador como el servidor de `gaia.cs.umass.edu` operan sobre HTTP/1.1, identificar el o los idiomas indicados en la cabecera `Accept-Language` de la petición, y leer el valor de `Content-Length` en la respuesta para determinar el tamaño exacto del contenido devuelto. Este ejercicio permitió pasar de conceptos teóricos de HTTP (formato de la línea de solicitud, código de estado, cabeceras) a su observación directa en tráfico real.

Conclusiones

1. Un equipo moderno cuenta con muchas más interfaces de red de las que un usuario típico percibe; la mayoría son virtuales o de soporte del sistema operativo (loopback, túneles VPN, funciones propias de Apple, puentes Thunderbolt) y no transportan tráfico de usuario, por lo que identificarlas correctamente es un paso necesario antes de capturar tráfico relevante.
2. El comportamiento de los navegadores modernos, caché persistente, peticiones condicionales y políticas de HTTPS-first, puede ocultar o alterar el tráfico que se busca analizar, por lo que un analista debe gestionar activamente el estado del navegador (limpieza de caché, ventanas nuevas, verificación de la URL final) para garantizar una captura representativa del protocolo estudiado.
3. Observar directamente los campos de una petición y respuesta HTTP (versión de protocolo, código de estado, cabeceras como Accept-Language y Content-Length) permite conectar la teoría del protocolo con su comportamiento real en una red, reforzando la comprensión conceptual mediante evidencia empírica.
4. Ante problemas de rendimiento en una red, es más práctico y apropiado capturar tráfico en múltiples puntos bajo control del analista (cliente, gateway, borde de red) que instalar herramientas de captura directamente en un servidor de producción, tanto por motivos técnicos (impacto en el rendimiento del propio servidor) como por consideraciones de acceso y privacidad.